



Universidade Estácio de Sá (UNESA)

Campus Estácio - Castelo - Belo Horizonte - MG

Desenvolvimento Full Stack

RELATÓRIO DO 2º PROCEDIMENTO DA MISSÃO PRÁTICA

Nível 2: Vamos Manter as Informações? (RPG0015) - Mundo 3

Turma: 9002 - Semestre: 2025.1

Aluno: Ivan de Ávila Carvalho Fleury Mortimer

2º Procedimento | Alimentando a Base

1. Objetivo da Prática

Este relatório tem como objetivo apresentar o processo de inserção de dados na base relacional criada anteriormente para o controle de operações de compra e venda de produtos. Todas as operações foram realizadas através do aplicativo Azure Data Studio conectado a um banco de dados SQL Server, operando dentro de um container Docker no macOS, uma vez que não é possível utilizar o SQL Server Management Studio em computadores Apple. O procedimento seguiu o roteiro da prática da disciplina.

2. Inserção de Dados

2.1 Inserção de Usuários

```
INSERT INTO dbo.Usuario (login, senha) VALUES  
('op1', 'op1'),  
('op2', 'op2');
```

Resultado da operação:

idUsuario	login	senha
1	op1	op1
2	op2	op2

2.2 Inserção de Produtos

```
INSERT INTO dbo.Produto (nome, quantidade, precoVenda) VALUES  
('Banana', 100, 5.00),  
('Pera', 50, 3.00),  
('Laranja', 500, 2.00),  
('Manga', 800, 4.00);
```

```
DELETE FROM dbo.Produto WHERE idProduto = 2;
```

Resultado final:

idProduto	nome	quantidade	precoVenda
1	Banana	100	5.00
3	Laranja	500	2.00
4	Manga	800	4.00

2.3 Inserção de Pessoas Físicas e Jurídicas

```
-- Pessoa Física
DECLARE @idPessoa INT = NEXT VALUE FOR dbo.Seq_idPessoa;

INSERT INTO dbo.Pessoa (idPessoa, nome, logradouro, cidade, estado, telefone,
email) VALUES
(@idPessoa, 'Joao', 'Rua 12, casa 3, Quitanda', 'Riacho do Sul', 'PA',
'1111-1111', 'joao@riacho.com');

INSERT INTO dbo.Pessoa_Fisica (idPessoa, cpf) VALUES (@idPessoa,
'11111111111');

-- Pessoa Jurídica
DECLARE @idPessoa2 INT = NEXT VALUE FOR dbo.Seq_idPessoa;

INSERT INTO dbo.Pessoa (idPessoa, nome, logradouro, cidade, estado, telefone,
email) VALUES
(@idPessoa2, 'JJC', 'Rua 11, Centro', 'Riacho do Norte', 'PA', '1212-1212',
'jjc@riacho.com');

INSERT INTO dbo.Pessoa_Juridica (idPessoa, cnpj) VALUES (@idPessoa2,
'222222222222');
```

Resultados finais:

• Tabela "Pessoa"

idPessoa	nome	logradouro	cidade	estado	telefone	Email
7	Joao	Rua 12, casa 3, Quitanda	Riacho do Sul	PA	1111-1111	joao@riacho.com
15	JJC	Rua 11, Centro	Riacho do Norte	PA	1212-1212	jjc@riacho.com

• Tabela "Pessoa_Fisica"

idPessoa	cpf
7	11111111111

• Tabela "Pessoa_Juridica"

idPessoa	cnpj
7	222222222222

3. Consultas SQL

a. Dados completos de pessoas físicas

```
SELECT pf.idPessoa, p.nome, p.logradouro, p.cidade, p.estado, p.telefone,  
p.email, pf.cpf  
FROM dbo.Pessoa_Fisica pf  
JOIN dbo.Pessoa p ON pf.idPessoa = p.idPessoa;
```

b. Dados completos de pessoas jurídicas

```
SELECT pj.idPessoa, p.nome, p.logradouro, p.cidade, p.estado, p.telefone,  
p.email, pj.cnpj  
FROM dbo.Pessoa_Juridica pj  
JOIN dbo.Pessoa p ON pj.idPessoa = p.idPessoa;
```

c. Movimentações de entrada, com produto, fornecedor, quantidade, preço unitário e valor total

```
SELECT m.idMovimento, pr.nome AS produto, p.nome AS fornecedor, m.quantidade,  
m.valorUnitario, (m.quantidade * m.valorUnitario) AS valorTotal  
FROM dbo.Movimento m  
JOIN dbo.Pessoa p ON m.idPessoa = p.idPessoa  
JOIN dbo.Produto pr ON m.idProduto = pr.idProduto  
WHERE m.tipo = 'E';
```

d. Movimentações de saída, com produto, comprador, quantidade, preço unitário e valor total

```
SELECT m.idMovimento, pr.nome AS produto, p.nome AS comprador, m.quantidade,  
m.valorUnitario, (m.quantidade * m.valorUnitario) AS valorTotal  
FROM dbo.Movimento m  
JOIN dbo.Pessoa p ON m.idPessoa = p.idPessoa  
JOIN dbo.Produto pr ON m.idProduto = pr.idProduto  
WHERE m.tipo = 'S';
```

e. Valor total das entradas agrupadas por produto

```
SELECT pr.nome AS produto, SUM(m.quantidade * m.valorUnitario) AS totalEntrada  
FROM dbo.Movimento m  
JOIN dbo.Produto pr ON m.idProduto = pr.idProduto  
WHERE m.tipo = 'E'  
GROUP BY pr.nome;
```

f. Valor total das saídas agrupadas por produto

```
SELECT pr.nome AS produto, SUM(m.quantidade * m.valorUnitario) AS totalSaida
FROM dbo.Movimento m
JOIN dbo.Produto pr ON m.idProduto = pr.idProduto
WHERE m.tipo = 'S'
GROUP BY pr.nome;
```

g. Operadores que não efetuaram movimentações de entrada (compra)

```
SELECT u.idUsuario, u.login
FROM dbo.Usuario u
WHERE u.idUsuario NOT IN (
    SELECT DISTINCT idUsuario FROM dbo.Movimento WHERE tipo = 'E'
);
```

h. Valor total de entrada, agrupado por operador

```
SELECT u.login, SUM(m.quantidade * m.valorUnitario) AS totalEntrada
FROM dbo.Movimento m
JOIN dbo.Usuario u ON m.idUsuario = u.idUsuario
WHERE m.tipo = 'E'
GROUP BY u.login;
```

i. Valor total de saída, agrupado por operador

```
SELECT u.login, SUM(m.quantidade * m.valorUnitario) AS totalSaida
FROM dbo.Movimento m
JOIN dbo.Usuario u ON m.idUsuario = u.idUsuario
WHERE m.tipo = 'S'
GROUP BY u.login;
```

j. Valor médio de venda por produto, utilizando média ponderada

```
SELECT p.nome,
    SUM(m.quantidade * m.valorUnitario) * 1.0 / NULLIF(SUM(m.quantidade),
0) AS mediaPonderada
FROM dbo.Movimento m
JOIN dbo.Produto p ON m.idProduto = p.idProduto
WHERE m.tipo = 'S'
GROUP BY p.nome;
```

4. Análise e Conclusão

Este segundo procedimento reforçou o uso de comandos DML (Data Manipulation Language) no SQL Server, permitindo a inserção, atualização e consulta de dados com consistência. As constraints e triggers implementadas garantiram a integridade das informações e evitaram inserções incorretas, como valores divergentes de preço ou saídas de estoque maiores que a disponibilidade. O uso do Azure Data Studio em ambiente macOS demonstrou ser uma alternativa eficaz para gestão do SQL Server sem o SSMS. Alguns pontos particularmente relevantes – no que tange a conceitos abordados e certas funções utilizadas ao longo da prática – são tratados nas respostas às perguntas que se seguem.

a. Quais as diferenças no uso de sequence e identity?

A principal diferença entre SEQUENCE e IDENTITY está na flexibilidade. O IDENTITY é atribuído automaticamente em uma coluna de tabela e gerenciado pelo SQL Server no momento da inserção, enquanto a SEQUENCE é um objeto separado que pode ser utilizado por múltiplas tabelas ou operações, oferecendo maior controle sobre a geração de valores.

b. Qual a importância das chaves estrangeiras para a consistência do banco?

As chaves estrangeiras garantem a integridade referencial entre tabelas, ou seja, asseguram que os valores inseridos em colunas relacionadas existam na tabela referenciada. Isso evita registros órfãos e mantém a coerência dos dados no banco relacional.

c. Quais operadores do SQL pertencem à álgebra relacional e quais são definidos no cálculo relacional?

Operadores como SELECT, PROJECT (projeção), JOIN, UNION, INTERSECT e DIFFERENCE são derivados da álgebra relacional. Já o cálculo relacional se refere a consultas baseadas em predicados lógicos, que são implementadas em SQL por meio de cláusulas como WHERE e EXISTS.

d. Como é feito o agrupamento em consultas, e qual requisito é obrigatório?

O agrupamento em consultas é feito por meio da cláusula GROUP BY, que organiza os dados em grupos com base em uma ou mais colunas. Um requisito obrigatório é que todas as colunas do SELECT, que não estão agregadas, devem ser incluídas na cláusula GROUP BY.

5. Repositório do Código

O projeto foi versionado e está disponível no GitHub no seguinte link:

https://github.com/ivanmortimer/Missao_Pratica_N2_M3_Vamos_Manter_as_Informacoes